

Database Management Systems – Pseudocode Answers

Q1. Complete Query Processing Pipeline

```
ALGORITHM QueryProcessingPipeline(sqlQuery, catalog)
INPUT : sqlQuery    -- raw SQL text typed by the user
        catalog     -- schema, indexes, table/column statistics
OUTPUT: resultSet   -- final tuples returned to the caller

BEGIN
    tokens          <- LexicalScan(sqlQuery)
    parseTree       <- SyntaxParse(tokens)
    IF parseTree is INVALID THEN
        RAISE SyntaxError("Malformed SQL")
    END IF

    FOR EACH relation R referenced in parseTree DO
        IF R NOT IN catalog THEN
            RAISE SemanticError("Unknown relation: " + R)
        END IF
    END FOR

    FOR EACH attribute A referenced in parseTree DO
        IF A NOT IN Schema(RelationOf(A)) THEN
            RAISE SemanticError("Unknown attribute: " + A)
        END IF
    END FOR

    logicalPlan <- TranslateToRelationalAlgebra(parseTree)

    logicalPlan <- PushSelectionsDown(logicalPlan)
    logicalPlan <- PushProjectionsDown(logicalPlan)
    logicalPlan <- EliminateRedundantOperators(logicalPlan)
    logicalPlan <- ReorderJoinsCandidates(logicalPlan)

    candidatePlans <- GeneratePhysicalPlans(logicalPlan, catalog)
    bestPlan       <- NIL
    minCost        <- INFINITY
    FOR EACH plan IN candidatePlans DO
        cost <- EstimateCost(plan, catalog)
        IF cost < minCost THEN
            minCost <- cost
            bestPlan <- plan
        END IF
    END FOR

    executionPlan <- GenerateExecutablePlan(bestPlan)

    executionPlan.Open()
    resultSet <- EMPTY LIST
    tuple <- executionPlan.Next()
    WHILE tuple != NULL DO
        APPEND tuple TO resultSet
        tuple <- executionPlan.Next()
    END WHILE
    executionPlan.Close()

    RETURN FormatOutput(resultSet)
END
```

Q2. Cost-Based Join Order Optimization

```
ALGORITHM CostBasedJoinOrdering(relations[1..n], stats)
INPUT : relations -- list of n relations to join
       stats      -- {pages(R), tuples(R), distinctValues(R.A), ...}
OUTPUT: bestPlan  -- join order + access paths with minimum estimated cost

BEGIN
    bestPlanFor <- EMPTY MAP

    FOR EACH R IN relations DO
        subset <- {R}
        bestPlanFor[subset] <- ChooseBestAccessPath(R, stats)
    END FOR

    FOR size = 2 TO n DO
        FOR EACH subset S of size 'size' from relations DO
            bestCost <- INFINITY
            bestJoinPlan <- NIL
            FOR EACH proper non-empty split (S1, S2) of S WHERE S1 UNION S2 = S DO
                planLeft <- bestPlanFor[S1]
                planRight <- bestPlanFor[S2]

                outCard <- EstimateJoinCardinality(planLeft, planRight, stats)

                FOR EACH joinAlgo IN {NestedLoop, BlockNestedLoop, SortMerge, HashJoin}
                    IF JoinPredicateExists(S1, S2) OR AllowCrossProduct THEN
                        ioCost <- EstimateJoinCost(planLeft, planRight, joinAlgo, stats)
                        totalCost <- planLeft.cost + planRight.cost + ioCost

                        IF totalCost < bestCost THEN
                            bestCost <- totalCost
                            bestJoinPlan <- BuildJoinNode(planLeft, planRight,
                                                            joinAlgo, outCard, totalCost)
                        END IF
                    END IF
                END FOR
            END FOR
            bestPlanFor[S] <- bestJoinPlan
        END FOR
    END FOR

    fullSet <- {relations[1], ..., relations[n]}
    bestPlan <- bestPlanFor[fullSet]
    RETURN bestPlan
END

FUNCTION EstimateJoinCost(left, right, algo, stats)
    SWITCH algo
        CASE NestedLoop:
            RETURN left.pages + (left.tuples * right.pages)
        CASE BlockNestedLoop:
            RETURN left.pages + CEIL(left.pages / B) * right.pages
        CASE SortMerge:
            RETURN SortCost(left) + SortCost(right) + (left.pages + right.pages)
        CASE HashJoin:
            RETURN 3 * (left.pages + right.pages)
    END SWITCH
END
```

Q3. Nested Loop Join

```
ALGORITHM NestedLoopJoin(R, S, theta, B)
INPUT : R      -- outer relation, bR pages, nR tuples
        S      -- inner relation, bS pages, nS tuples
        theta   -- join predicate
        B      -- number of available buffer pages (B >= 3: 1 out, 1 in for S, rest for
R)
OUTPUT: joinResult -- set of tuples satisfying theta

BEGIN
    joinResult <- EMPTY SET
    outBuffer  <- allocate 1 page for output

    FOR EACH block Br OF (B - 2) PAGES of R (read into memory) DO
        FOR EACH page Ps OF S DO
            LOAD Ps INTO memory
            FOR EACH tuple r IN Br DO
                FOR EACH tuple s IN Ps DO
                    IF theta(r, s) = TRUE THEN
                        APPEND (r JOIN s) TO outBuffer
                        IF outBuffer IS FULL THEN
                            FLUSH outBuffer TO joinResult
                            CLEAR outBuffer
                        END IF
                    END IF
                END FOR
            END FOR
        END FOR
    END FOR

    IF outBuffer IS NOT EMPTY THEN
        FLUSH outBuffer TO joinResult
    END IF

    RETURN joinResult
END
```

Q4. Deadlock Detection using Wait-For Graph (WFG)

```
ALGORITHM DetectDeadlock(transactions, lockTable)
INPUT : transactions -- active transactions T1..Tn
        lockTable    -- current lock holders & waiters per data item
OUTPUT: hasDeadlock, cycleTransactions

BEGIN
    WFG <- EMPTY GRAPH

    FOR EACH dataItem D IN lockTable DO
        holders <- TransactionsHolding(D)
        waiters  <- TransactionsWaitingFor(D)
        FOR EACH Ti IN waiters DO
            FOR EACH Tj IN holders DO
                IF Ti != Tj THEN
                    AddEdge(WFG, Ti, Tj)
                END IF
            END FOR
        END FOR
    END FOR

    visited <- EMPTY SET
    onStack <- EMPTY SET
```

```

parentOf <- EMPTY MAP

FOR EACH node T IN WFG.nodes DO
    IF T NOT IN visited THEN
        (found, cycle) <- DFS_DetectCycle(T, WFG, visited, onStack, parentOf)
        IF found = TRUE THEN
            RETURN TRUE, cycle
        END IF
    END IF
END FOR

RETURN FALSE, EMPTY SET
END

FUNCTION DFS_DetectCycle(node, WFG, visited, onStack, parentOf)
    ADD node TO visited
    ADD node TO onStack

    FOR EACH neighbor IN WFG.AdjacentTo(node) DO
        IF neighbor NOT IN visited THEN
            parentOf[neighbor] <- node
            (found, cycle) <- DFS_DetectCycle(neighbor, WFG, visited, onStack, parentOf)
            IF found = TRUE THEN
                RETURN TRUE, cycle
            END IF
        ELSE IF neighbor IN onStack THEN
            cycle <- ReconstructCycle(neighbor, node, parentOf)
            RETURN TRUE, cycle
        END IF
    END FOR

    REMOVE node FROM onStack
    RETURN FALSE, EMPTY SET
END

FUNCTION ReconstructCycle(startNode, endNode, parentOf)
    cycle <- [endNode]
    current <- endNode
    WHILE current != startNode DO
        current <- parentOf[current]
        PREPEND current TO cycle
    END WHILE
    RETURN cycle
END

```

Q5. Basic Two-Phase Locking (2PL) Protocol

```

ALGORITHM BasicTwoPhaseLocking(transaction T, operationList)
INPUT : T          -- transaction requesting operations
        operationList -- sequence of Read(X) / Write(X) requests
OUTPUT: T commits (all locks released) or is aborted/rolled back

```

```

BEGIN
    phase <- GROWING
    lockSet <- EMPTY SET

    FOR EACH operation OP IN operationList DO

        IF OP is Read(X) THEN
            IF phase != GROWING THEN
                RAISE ProtocolViolation("Cannot acquire lock in shrinking phase")
            END IF

```

```

        RequestLock(T, X, SHARED)

    ELSE IF OP is Write(X) THEN
        IF phase != GROWING THEN
            RAISE ProtocolViolation("Cannot acquire lock in shrinking phase")
        END IF
        RequestLock(T, X, EXCLUSIVE)
    END IF
END FOR

Commit(T)
FOR EACH lock IN lockSet DO
    ReleaseLock(T, lock)
END FOR
END

FUNCTION RequestLock(T, X, mode)
    LOOP
        IF LockTable[X].isCompatible(mode, currentHolders(X)) THEN
            GRANT lock(X, mode) TO T
            ADD (X, mode) TO lockSet(T)
            RETURN GRANTED
        ELSE
            ADD T TO WaitQueue(X)
            BLOCK T UNTIL awakened
        END IF
    END LOOP
END

FUNCTION ReleaseLock(T, (X, mode))
    phase <- SHRINKING
    REMOVE T FROM currentHolders(X)
    REMOVE (X, mode) FROM lockSet(T)
    WAKE UP transactions IN WaitQueue(X) whose requested mode is now compatible
END

```

Q6. Timestamp-Ordering Concurrency Control with Thomas' Write Rule

```

ALGORITHM TimestampOrderingWithThomasWriteRule(T, operation, X)
INPUT : T          -- transaction with timestamp TS(T)
       operation   -- Read or Write
       X          -- data item, with RTS(X), WTS(X)
OUTPUT: operation executed, or T aborted & restarted with a new timestamp

BEGIN
    IF operation = Read(X) THEN
        IF TS(T) < WTS(X) THEN
            ABORT T
            RESTART T WITH new timestamp
        ELSE
            EXECUTE Read(X)
            RTS(X) <- MAX(RTS(X), TS(T))
        END IF

    ELSE IF operation = Write(X) THEN
        IF TS(T) < RTS(X) THEN
            ABORT T
            RESTART T WITH new timestamp

        ELSE IF TS(T) < WTS(X) THEN

```

```

        IGNORE Write(X)

    ELSE
        EXECUTE Write(X)
        WTS(X) <- TS(T)
    END IF
END IF
END

```

Q7. ARIES Recovery Algorithm (Analysis, Redo, Undo)

```

ALGORITHM ARIESRecovery(log, lastCheckpoint)
INPUT : log          -- write-ahead log records (LSN, type, transID, pageID, undo/redo
info)
        lastCheckpoint -- most recent checkpoint record
OUTPUT: database restored to consistent state

BEGIN
    dirtyPageTable <- CopyFrom(lastCheckpoint.DPT)
    transactionTable <- CopyFrom(lastCheckpoint.TT)

    FOR EACH logRecord FROM lastCheckpoint.LSN TO END OF LOG DO
        IF logRecord.type IN {UPDATE, COMPENSATION} THEN
            IF logRecord.pageID NOT IN dirtyPageTable THEN
                dirtyPageTable[logRecord.pageID] <- logRecord.LSN
            END IF
        END IF

        IF logRecord.type = UPDATE OR logRecord.type = BEGIN THEN
            transactionTable[logRecord.transID].lastLSN <- logRecord.LSN
            transactionTable[logRecord.transID].status <- ACTIVE
        ELSE IF logRecord.type = COMMIT THEN
            transactionTable[logRecord.transID].status <- COMMITTED
        ELSE IF logRecord.type = END THEN
            REMOVE logRecord.transID FROM transactionTable
        END IF
    END FOR

    losers <- {t IN transactionTable | t.status != COMMITTED}

    redoStartLSN <- MIN(recLSN FOR EACH page IN dirtyPageTable)

    FOR EACH logRecord FROM redoStartLSN TO END OF LOG DO
        IF logRecord.type IN {UPDATE, COMPENSATION} THEN
            P <- logRecord.pageID
            IF P IN dirtyPageTable AND logRecord.LSN >= dirtyPageTable[P] THEN
                IF PageLSN(P) < logRecord.LSN THEN
                    REDO the action described in logRecord ON page P
                    PageLSN(P) <- logRecord.LSN
                END IF
            END IF
        END IF
    END FOR

    toUndo <- PriorityQueue ordered by LSN DESCENDING
        INIT WITH lastLSN of every transaction IN losers

    WHILE toUndo IS NOT EMPTY DO
        currentLSN <- Pop largest LSN FROM toUndo
        logRecord <- ReadLog(currentLSN)

        IF logRecord.type = UPDATE AND logRecord.isUndoable THEN
            UNDO the action (restore before-image) ON logRecord.pageID
        END IF
    END WHILE

```

```

        WRITE a Compensation Log Record (CLR) with:
            undoNextLSN <- logRecord.prevLSN
        PUSH logRecord.prevLSN ONTO toUndo (if not NULL and belongs to a loser)

    ELSE IF logRecord.type = COMPENSATION THEN
        IF logRecord.undoNextLSN != NULL THEN
            PUSH logRecord.undoNextLSN ONTO toUndo
        END IF

    ELSE IF logRecord.type = BEGIN THEN
        WRITE an END log record for logRecord.transID
        REMOVE logRecord.transID FROM losers
    END IF
END WHILE

RETURN "Database recovered: all committed changes durable, all loser effects undone"
END

```

Q8. Shadow Paging Recovery

```

ALGORITHM ShadowPagingRecovery(transaction T, writeOperations)
INPUT : T          -- transaction to execute
        writeOperations -- sequence of page writes requested by T
OUTPUT: database durably updated on commit, or unaffected on crash/abort

BEGIN
    shadowPageTable <- COPY OF currentPageTable
    workingPageTable <- currentPageTable

    FOR EACH writeOp(pageID, newData) IN writeOperations DO
        oldFrame <- workingPageTable[pageID]

        newFrame <- AllocateNewDiskBlock()
        WRITE newData TO newFrame

        workingPageTable[pageID] <- newFrame
    END FOR

    IF T decides to COMMIT THEN
        FLUSH all newFrame pages TO disk
        FLUSH workingPageTable TO disk AS a NEW page table

        ATOMICALLY: currentPageTableLocation <- location of workingPageTable

        DEALLOCATE all oldFrame pages that are no longer
                     referenced by workingPageTable (garbage collect old shadow blocks)

    ELSE
        DEALLOCATE all newFrame pages allocated during T's execution
        workingPageTable <- DISCARD (currentPageTable already = shadowPageTable,
untouched)
    END IF

    RETURN "Transaction " + (COMMITTED or ROLLED BACK)
END

FUNCTION RecoverAfterCrash()
    IF currentPageTableLocation was NEVER atomically updated to point to
       the new (working) page table THEN
        currentPageTable <- shadowPageTable
    ELSE
        currentPageTable <- (the already-updated table)
    END IF
END FUNCTION

```

```

    END IF
    RETURN "Database consistent -- no REDO/UNDO log processing required"
END

```

Q9. Immediate Update Recovery (UNDO/REDO log-based)

ALGORITHM ImmediateUpdateRecovery(log)

INPUT : log -- sequence of log records:

```

    <START, T>
    <T, X, old_value, new_value>  -- update record
    <COMMIT, T>
    <ABORT, T>

```

OUTPUT: database consistent (equivalent to a serial execution of only committed transactions)

BEGIN

```

    committedTxns <- EMPTY SET
    activeTxns    <- EMPTY SET

```

FOR EACH record IN log (in the order written) DO

SWITCH record.type

CASE START:

ADD record.T TO activeTxns

CASE COMMIT:

ADD record.T TO committedTxns

REMOVE record.T FROM activeTxns

CASE ABORT:

REMOVE record.T FROM activeTxns

END SWITCH

END FOR

toUndo <- activeTxns

FOR EACH record IN log, TRAVERSED FROM END TO START DO

IF record.type = UPDATE AND record.T IN toUndo THEN

WRITE record.X <- record.old_value

APPEND <T, X, old_value> TO log AS COMPENSATION (CLR)

END IF

IF record.type = START AND record.T IN toUndo THEN

APPEND <ABORT, record.T> TO log

REMOVE record.T FROM toUndo

END IF

END FOR

FOR EACH record IN log (in the order written) DO

IF record.type = UPDATE AND record.T IN committedTxns THEN

WRITE record.X <- record.new_value

END IF

END FOR

FLUSH ALL modified pages TO disk

WRITE <CHECKPOINT> TO log

RETURN "Recovery complete: committed transactions REDONE, incomplete transactions UNDONE"

END

Q10. Database Connectivity Manager (Connection Pooling, Execution, Rollback)

CLASS DatabaseConnectivityManager


```

ATTRIBUTES:
    connectionPool    -- list of available (idle) physical connections
    activeConnections -- set of connections currently checked out
    maxPoolSize       -- upper bound on total physical connections
    minPoolSize       -- connections kept warm even when idle
    waitQueue         -- requests waiting when the pool is exhausted

FUNCTION Initialize(dbConfig)
    connectionPool <- EMPTY LIST
    FOR i = 1 TO minPoolSize DO
        conn <- OpenPhysicalConnection(dbConfig)
        ADD conn TO connectionPool
    END FOR
END FUNCTION

FUNCTION GetConnection()
    IF connectionPool IS NOT EMPTY THEN
        conn <- REMOVE first item FROM connectionPool
    ELSE IF SIZE(activeConnections) < maxPoolSize THEN
        conn <- OpenPhysicalConnection(dbConfig)
    ELSE
        ADD current request TO waitQueue
        BLOCK UNTIL a connection is released OR timeout occurs
        IF timeout occurs THEN
            RAISE ConnectionTimeoutException
        END IF
        conn <- assigned connection FROM waitQueue notification
    END IF

    IF NOT IsValid(conn) THEN
        CLOSE conn
        conn <- OpenPhysicalConnection(dbConfig)
    END IF

    ADD conn TO activeConnections
    RETURN conn
END FUNCTION

FUNCTION ExecuteQuery(sql, params)
    conn <- GetConnection()
    result <- NULL

    TRY
        conn.BeginTransaction()
        preparedStmt <- conn.Prepare(sql)
        BIND params TO preparedStmt

        IF sql IS a SELECT statement THEN
            result <- preparedStmt.ExecuteQuery()
        ELSE
            rowsAffected <- preparedStmt.ExecuteUpdate()
            result <- rowsAffected
        END IF

        conn.Commit()

    CATCH DatabaseException AS ex
        conn.Rollback()
        LOG ex
        RAISE ex TO caller

    CATCH ConnectionLostException AS ex
        conn.Rollback()

```

```

        DiscardConnection(conn)
        conn <- NULL
        RAISE ex TO caller

    FINALLY
        IF conn != NULL THEN
            ReleaseConnection(conn)
        END IF
    END TRY

    RETURN result
END FUNCTION

FUNCTION ReleaseConnection(conn)
    REMOVE conn FROM activeConnections
    IF waitQueue IS NOT EMPTY THEN
        nextRequest <- POP FROM waitQueue
        ASSIGN conn TO nextRequest
    ELSE
        RESET conn state (clear temp settings)
        ADD conn TO connectionPool
    END IF
END FUNCTION

FUNCTION DiscardConnection(conn)
    TRY
        CLOSE conn
    CATCH ANY
    END TRY
END FUNCTION

FUNCTION Shutdown()
    FOR EACH conn IN connectionPool DO
        CLOSE conn
    END FOR
    FOR EACH conn IN activeConnections DO
        conn.Rollback()
        CLOSE conn
    END FOR
END FUNCTION
END CLASS

```